Nesneye Dayalı Programlama

Hafta 5

Prof. Dr. Ümit KOCABIÇAK
Dr.Öğr.Üyesi Hüseyin Eski
Özgür ÇİFTÇİ

Bu ders içeriğinin basım, yayım ve satış hakları Sakarya Üniversitesi'ne aittir. "Uzaktan Öğretim" tekniğine uygun olarak hazırlanan bu ders içeriğinin bütün hakları saklıdır. İlgili kuruluştan izin almadan ders içeriğinin tümü ya da bölümleri mekanik, elektronik, fotokopi, manyetik kayıt veya başka şekillerde çoğaltılamaz, basılamaz ve dağıtılamaz.
Her hakkı saklıdır © 2010 Sakarya Üniversitesi

5.1. NDP Temel Kavram II: Kalıtım (Inheritance)

Tıpkı doğada olduğu gibi (anne-babanın çocuklarına genetik özelliklerini devredebilmesi); kalıtım bir sınıfın izin verilen üyelerini başka sınıflara verebilme yeteneği olarak tanımlanabilir. (Bir sınıfın başka bir sınıfın üyelerini kendi üyesi gibi davranmalarını sağlayabilen özellik.) Burada ki temel amaç benzer kodları barındıran sınıfların ortak özellik/davranışlarını bir araya toplayarak kodun tekrar kullanılabilirliği (reuse) ve merkezi yönetimini sağlamaktır.

Burada kalıtım veren sınıf; taban(base), kalıtım alan sınıf ise türemiş sınıf (derived, çocuk sınıf) olarak adlandırılır. Taban sınıftan türemiş sınıfa doğru kalıtım verme yada tam aksi olarak türemiş sınıftan taban sınıfa doğru ise miras alma gerçekleşir.

Türemiş sınıflar, taban sınıfların özelliklerini/davranışlarını alabildikleri gibi kendileri başka özellikler edinebilir, yada almış oldukları özellik/davranışları geçersiz kılabilirler.

C++ birden çok sınıftan kalıtım almayı (multi inheritance) desteklerken, C# ve Java'da buna kısıtlama getirerek ancak tek bir sınıftan kalıtım almayı, ancak daha fazla gerek duyulan hallerde interface (arayüz) adı verilen yapılardan mümkün kılmıştır. Sınıflardan kalıtım C#'ta : operatörü ile sağlanırken Java'da extends anahtar kelimesi ile gerçekleştirilir.

Teorik olarak kalıtım sonsuz seviyededir. Ancak tecrübeler 4 ila 6. kademeden sonra analizin ciddi şekilde tekrar incelenmesi gerektiğini söylemektedir.

Temel Sınıf

```
class BaseClass
  {
    public void EkranaYazdirBase()
    {
      Console.WriteLine("Temel Sınıf");
    }
  }
```

Türemiş Sınıfa temel sınıftan kalıtım veriyoruz.

```
class DerivedClass:BaseClass
  {
    public void EkranaYazdirDerived()
    {
      Console.WriteLine("Türemiş Sınıf");
    }
  }
```

```
class Program
{
  static void Main(string[] args)
  {
    BaseClass bc = new BaseClass();

    bc.EkranaYazdirBase();

    DerivedClass dc = new DerivedClass();
    dc.
  }
}
```

EkranaYazdirBase
EkranaYazdirDerived
Equals
GetHashCode
GetType
ToString

void BaseClass.EkranaYazdirBase()

DerivedClass'ta her ne kadar EkranaYazdirBase() isimli bir metot olmasa da BaseClass'tan kalıtım aldığından dolayı izin verilen tüm özellikler/davranışlar türeyen sınıfa geçmektedir.

Main kodları

```
class Program
  {
    static void Main(string[] args)
    {
      BaseClass bc = new BaseClass();

      bc.EkranaYazdirBase();

      DerivedClass dc = new DerivedClass();
      dc.EkranaYazdirBase();
      dc.EkranaYazdirDerived();
    }
  }
```

C:\Windows\system32\cmd.exe

```
Temel Sınıf
Temel Sınıf
Türemiş Sınıf
Devam etmek için bir tuşa basın . . .
```

5.1.1. Erişim Denetliyicilerin Kalıtım üzerindeki etkileri

```
class BaseClass
  {
    public string adi;
    protected string soyAdi;
    private string meslek;
  }
```

```
class DerivedClass : BaseClass
  {
    public void EkranaYazdir()
    {

    }

  }
```

public ile izin verilen alanlar alt sınıfta kullanılabilmektedir.

```
class DerivedClass : BaseClass
{
  public void EkranaYazdir()
  {
    Console.WriteLine("Adi" + a

  }

}
```

ActivationContext
Activator
adi
AggregateException
AppDomain
AppDomainInitializer
AppDomainManager
AppDomainManagerInitializationOptic
AppDomainSetup

string BaseClass.adi

protected ile izin verilen alanlar alt sınıfta kullanılabilmektedir.

```
class DerivedClass : BaseClass
{
   public void EkranaYazdir()
   {
      Console.WriteLine("Adi" + adi);
      Console.WriteLine("Soyadi" + s
                                     sim
                                     Single
                                     sizeof
                                     SortedDictionary<>
                                     SortedList<>
                                     SortedSet<>
                                     soyAdi        string BaseClass.soyAdi
                                     Stack<>
                                     stackalloc
   }

}
```

Temel sınıfta private olarak tanımlanan değerler alt sınıfta kullanılamamaktadır.

```
class DerivedClass : BaseClass
{
   public void EkranaYazdir()
   {
      Console.WriteLine("Adi" + adi);
      Console.WriteLine("Soyadi" + soyAdi);
      Console.WriteLine("Mesleği" + mes
                                     InsufficientMemoryException
                                     MemberAccessException
                                     MemberwiseClone
   }                                 MethodAccessException
                                     MissingMemberException
}                                    MissingMethodException
                                     OutOfMemoryException
                                     ParallelMergeOptions
```

Temel sınıfta public olarak tanımlanan alanlar türemiş sınıf üzerinden diğer sınıflarda da kullanılabilmektedir.

```
class Program
{
  static void Main(string[] args)
  {
    DerivedClass dr = new DerivedClass();
    dr.
  }
}
```

adi
Equals
GetHashCode
GetType
ToString

5.2. base anahtar kelimesi

Türemiş sınıftan temel sınıf üyelerine (alanlar, kurucular, metotlar vb.) base anahtar kelimesi ile erişebilinir.

BaseClass

```
class BaseClass
  {
    public double a = 10;
    protected double b = 20;
    private double c = 30;
  }
```

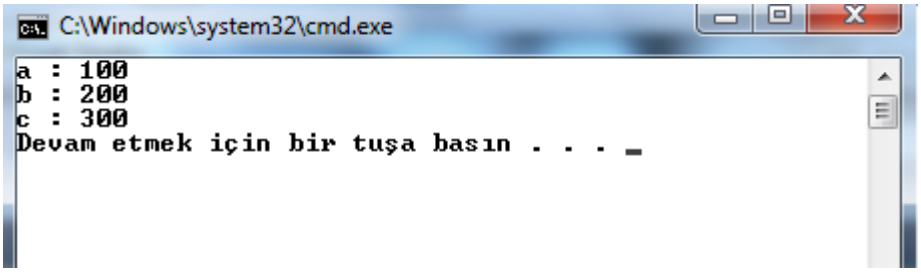
DerivedClass

```
class DerivedClass : BaseClass
  {

    private double a = 100;
    public double b = 200;
    protected double c = 300;

    public void EkranaYazdir()
    {
      Console.WriteLine("a : " + a);
      Console.WriteLine("b : " + b);
      Console.WriteLine("c : " + c);

    }
  }
```

Main kodları;

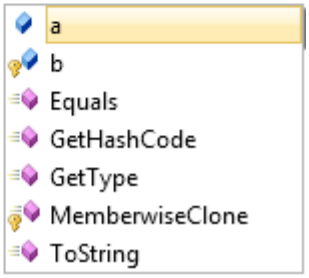
```
class Program
  {
    static void Main(string[] args)
    {
      DerivedClass dr = new DerivedClass();
      dr.EkranaYazdir();
    }
  }
```

Ekran çıktısı;



Hem temel sınıfta hemde türemiş sınıfta aynı isimdeki değişkenler var ise erişim denetliyicileri değişse bile en son değer olarak türemiş sınıftaki değerlerini almaktadır. Bu tür durumlarda bazen ana sıfıntaki değerleri ulaşılmak istenelirse base anahtar kelimesi kullanılara temel sınıfın izin verilen üyelerine ulaşılabilir.

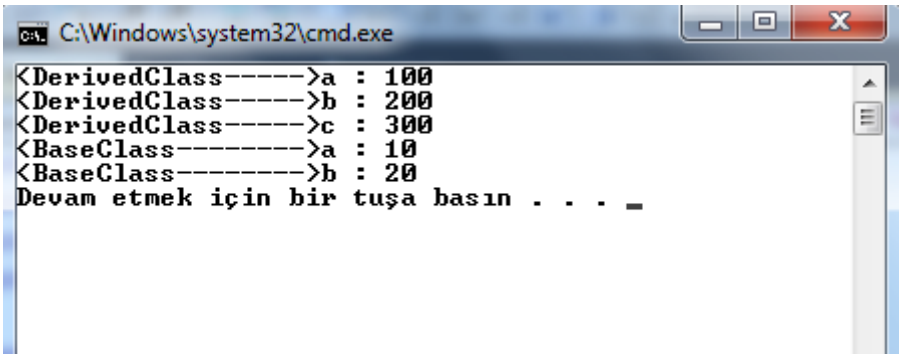
```
class DerivedClass : BaseClass
{

  private double a = 100;
  public double b = 200;
  protected double c = 300;

  public void EkranaYazdir()
  {
    Console.WriteLine("a : " + a);
    Console.WriteLine("b : " + b);
    Console.WriteLine("c : " + c);
    Console.WriteLine("a : " + base.

  }
}
```



DerivedClass kodlarımız;

```
class DerivedClass : BaseClass
  {

    private double a = 100;
    public double b = 200;
    protected double c = 300;

    public void EkranaYazdir()
    {
      Console.WriteLine("<DerivedClass----->a : " + a);
      Console.WriteLine("<DerivedClass----->b : " + b);
      Console.WriteLine("<DerivedClass----->c : " + c);
      Console.WriteLine("<BaseClass-------->a : " + base.a);
      Console.WriteLine("<BaseClass-------->b : " + base.b);

    }
  }
```

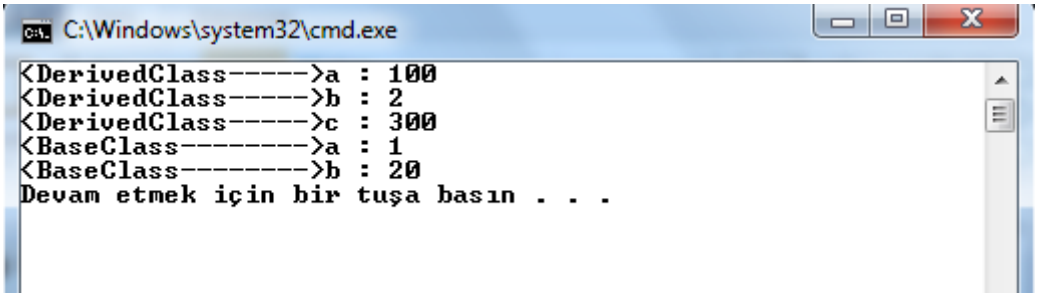
Ekran çıktısı;



Main kodlarımızı aşağıdaki şekilde değiştirelim.

```
class Program
  {
    static void Main(string[] args)
    {
      BaseClass br = new BaseClass();
      br.a = 1000;

      DerivedClass dr = new DerivedClass();
      dr.a = 1;
      dr.b = 2;
      dr.EkranaYazdir();
    }
  }
```

Ekran çıktımız;



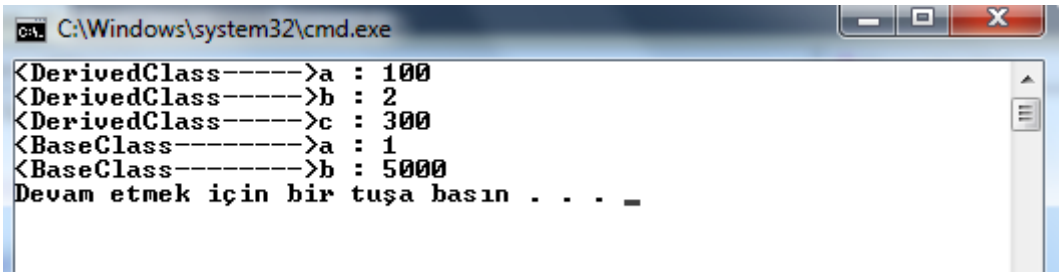
Bazı değerler değişirken bazıları değişmemiştir. Bunun nedenlerini incelersek;

Sınıf Adı	Alan	Eski Değer	Yeni Değer	Sebep
DerivedClass	a	100	100	DerivedClass'ta private olarak tanımlandığından değiştirilmeye çalışılan BaseClass'ın a değeri.
DerivedClass	b	200	2	DerivedClass'ta public olarak tanımlandığından değişme var.
DerivedClass	c	300	-	DerivedClass'ta protected olarak tanımlandığından main içersinden erişilemez.
BaseClass	a	1000	1	İlk etapta 1000 değeri atansa da BaseClass'taki özellikler DerivedClass'a geçmektedir. DerivedClass'tan üretilen yeni nesne ile tekrar değer verilmektedir.
BaseClass	b	20	20	BaseClass'ta protected olarak tanımlı olduğundan değeri sadece DerivedClass'tan değiştirilebilir.
BaseClass	c	30	-	BaseClass'ta private olarak tanımlı

DerivedClass kodlarımıza base.b=5000 ifadesini ekleyelim;

```
class DerivedClass : BaseClass
  {

    private double a = 100;
    public double b = 200;
    protected double c = 300;


    public void EkranaYazdir()
    {
      base.b = 5000;
      Console.WriteLine("<DerivedClass----->a : " + a);
      Console.WriteLine("<DerivedClass----->b : " + b);
      Console.WriteLine("<DerivedClass----->c : " + c);
      Console.WriteLine("<BaseClass-------->a : " + base.a);
      Console.WriteLine("<BaseClass-------->b : " + base.b);

    }
  }
```

BaseClass b değeri 5000 olarak yazdırılacaktır.



5.3. Metotlarda kalıtım

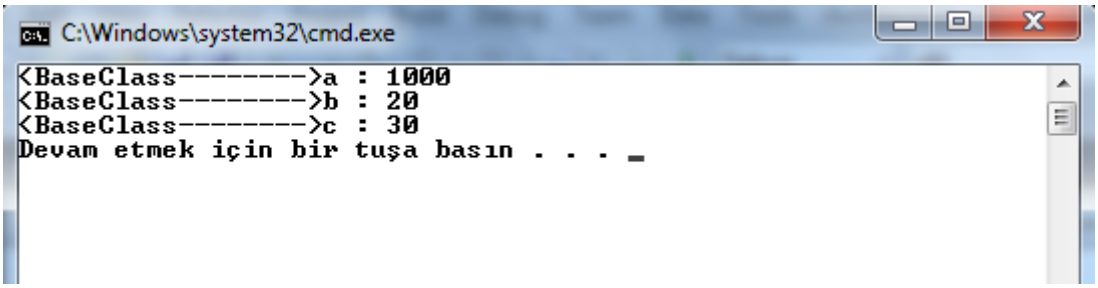
BaseClass

```
class BaseClass
  {
    public double a = 10;
    protected double b = 20;
    private double c = 30;

    public void EkranaYazdirBaseClass()
    {
      Console.WriteLine("<BaseClass-------->a : " + a);
      Console.WriteLine("<BaseClass-------->b : " + b);
      Console.WriteLine("<BaseClass-------->c : " + c);
    }
  }
```

Main kodları

```
class Program
  {
    static void Main(string[] args)
    {
      BaseClass br = new BaseClass();
      br.a = 1000;
      br.EkranaYazdirBaseClass();
    }
  }
```

a değişkenine en son 1000 değeri atandığından sonuç ekranında değeri 1000 olarak gözükmektedir.



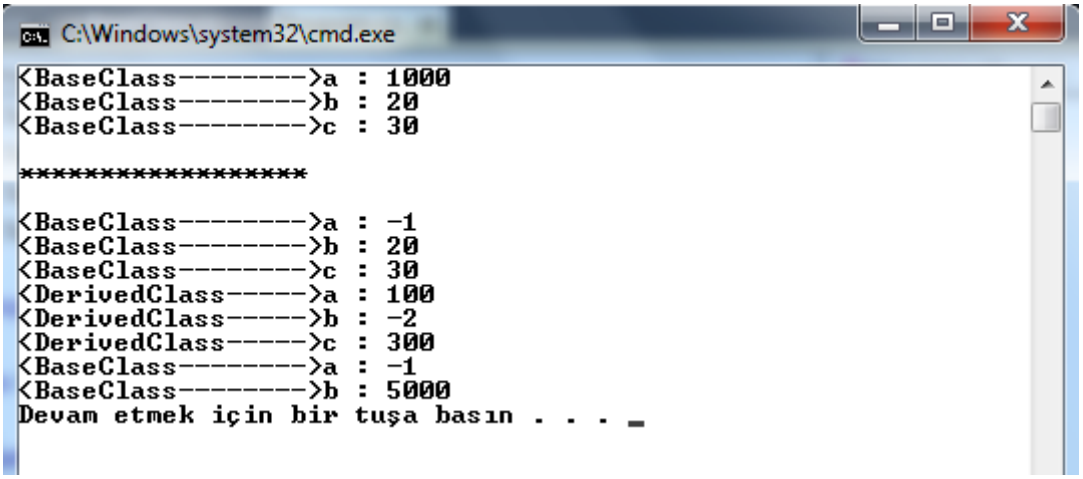
DerivedClass kodlarımız;

```
class DerivedClass : BaseClass
  {

    private double a = 100;
    public double b = 200;
    protected double c = 300;


    public void EkranaYazdirDerivedClass()
    {
      base.b = 5000;
      Console.WriteLine("<DerivedClass----->a : " + a);
      Console.WriteLine("<DerivedClass----->b : " + b);
      Console.WriteLine("<DerivedClass----->c : " + c);
      Console.WriteLine("<BaseClass-------->a : " + base.a);
      Console.WriteLine("<BaseClass-------->b : " + base.b);

    }
  }
```

Main kodlarının yeni hali;

```
class Program
  {
    static void Main(string[] args)
    {
      BaseClass br = new BaseClass();
      br.a = 1000;
      br.EkranaYazdirBaseClass();

      Console.WriteLine();
      Console.WriteLine("******************");
      Console.WriteLine();

      DerivedClass dr = new DerivedClass();
      dr.a = -1;
      dr.b = -2;
      dr.EkranaYazdirBaseClass(); // kalıtım yoluyla geçmiştir
      dr.EkranaYazdirDerivedClass();
    }
  }
```

Ekran görünümüzü ise aşağıdaki şekildedir.



Temel sınıfta ve türeyen sınıfta aynı isimde metotların bulunması durumunda ise türemiş sınıf temel sınıfın metotlarını geçersiz kılarak kendi tanımlamalarını kullanır. Ancak derleyici size yinede uyarı verecektir.

TemelSinif;

```
class TemelSinif
  {
    public void Hesapla()
    {
      Console.WriteLine("TemelSinif");
    }
  }
```

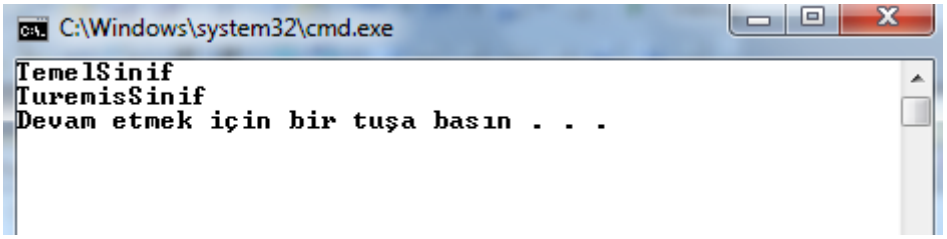
TuremisSinif

```
class TuremisSinif:TemelSinif
  {
    public void Hesapla()
    {
      Console.WriteLine("TuremisSinif");
    }
  }
```

Main Kodları

```
class Program
  {
    static void Main(string[] args)
    {
      TemelSinif ts = new TemelSinif();
      ts.Hesapla();

      TuremisSinif tus = new TuremisSinif();
      tus.Hesapla();
    }
  }
```

Ekran çıktısı;



Derleyicin vereceği bu uyarıyı türeyen sınıftaki metodun başına new anahtar sözcüğü getirerek aşabiliriz.

```
class TuremisSinif:TemelSinif
  {
    new public void Hesapla()
    {
      Console.WriteLine("TuremisSinif");
    }
  }
```

Metotların abstract yada virtual olması farklı bir durumdur.

5.3. Kurucularda Kalıtım

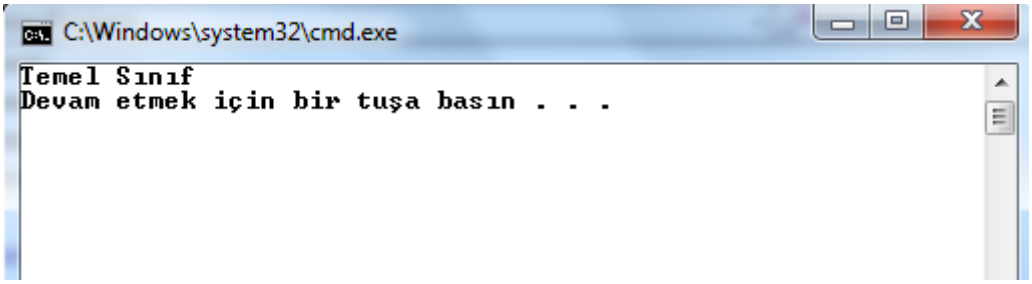
Durum 1:

A (temel sınıf) sınıfı;

```
class A
  {
    public A()
    {
      Console.WriteLine("Temel Sınıf");
    }
  }
```

```
class B:A
  {
  }
```

```
class Program
  {
    static void Main(string[] args)
    {
      B b = new B();
    }
  }
```



Sonuç: Türemiş sınıflardan bir nesne oluşturulurken arka planda kalıtım veren sınıftan (base class) otomatik olarak bir nesne oluşturulur ve kalıtım bu nesne üzerinden gerçekleştirilir. Burada dikkat edilmesi gereken konu türemiş sınıfın kurucusunun çalışmasından önce temel sınıfın kurucusunun çalışması gerektiğidir.

Durum 2: Türemiş sınıf temel sınıf kurucu yapılandırması.

```
class A
  {
    public A(int a, int b)
    {
      Console.WriteLine(a + b);
    }
  }
```

```
class B:A
  {
    public B(int a)
    {
      Console.WriteLine(a);
    }
  }
```

```
class Program
  {
    static void Main(string[] args)
    {
      B b = new B(5);
    }
  }
```

Error List

1 Error | 0 Warnings | 0 Messages

	Description	File
1	'Kurucular.A' does not contain a constructor that takes 0 arguments	B.cs

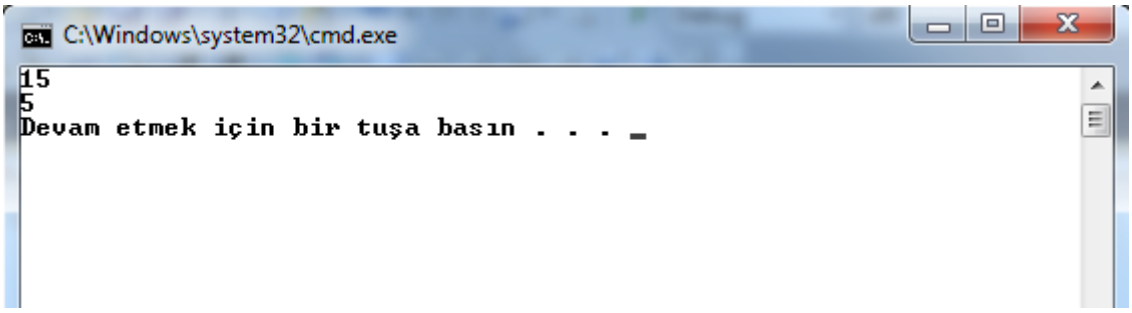
B sınıfı kurulurken A sınıfından kalıtım aldığından A sınıfının kurucusu çağrılmaktadır. Ancak A sınıfının kurucusunun istediği şartlar sağlanamamaktadır. A sınıfından otomatik olarak oluşacak nesnenin kurucusunun istediği başlangıç değerleri B sınıfından çağrılacak kurucudan sağlanmak zorundadır. Bu durumda B sınıfının kurucuları A sınıfının kurucularına base anahtar kelimesi kullanarak gerekli şartları sağlayabilirler.

Yukarıdaki örneği aşağıdaki biçimde değiştirelim.

```
class A
  {
    public A(int a, int b)
    {
      Console.WriteLine(a + b);
    }
  }
```

```
class B:A
  {

    public B(int a):base(a,10)
    {
      Console.WriteLine(a);
    }
  }
```

```
class Program
  {
    static void Main(string[] args)
    {
      B b = new B(5);
    }
  }
```



Kalıtımda başka bir konu ise kimin kimden türeyeceğidir. Uygulamalarda genel kabul; kurucudaki parametre sayısının fazla olanının temel sınıf olması yönündedir.

5.4. Kalıtımın engellenmesi

NDP de her sınıf diğer sınıf yada arayüzlerden[1] kalıtım almabilmekteydi. Bazı durumda bunun engellenmesi gerekebilir. C#'ta sealed anahtar kelimesi ile bir sınıfın kalıtım vermesi engellenebilir. (Static[2] sınıflar doğası gereği kalıtım almazlar ve vermezler.)

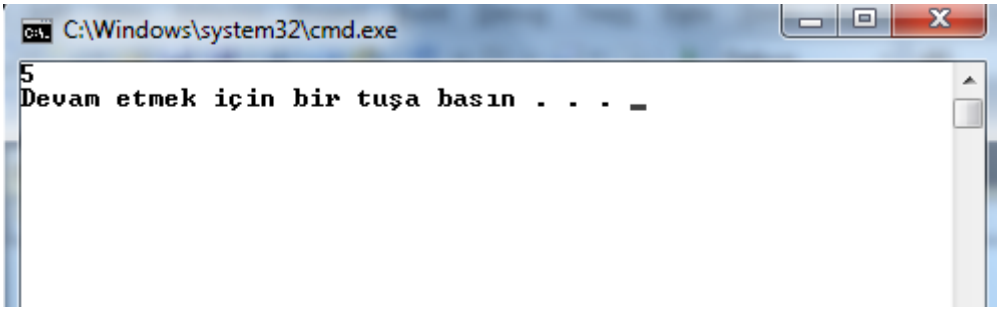
```
public class Calisan
  {
    public string adi=null;
    public string soyadi=null;

  }
```

```
public class Mudur:Calisan
  {
    public string departman=null;

  }
```

```
sealed public class GenelMudur:Mudur
  {

    public double MaasHesapla()
    {
      return 5;
    }
  }
```

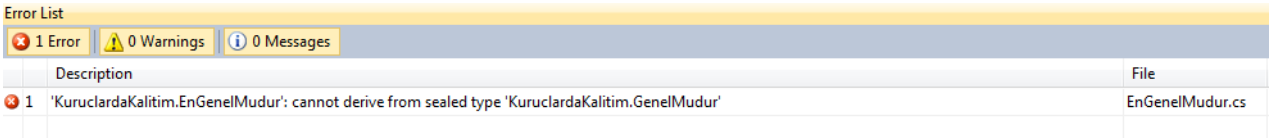
```
class Program
  {
    static void Main(string[] args)
    {
      Mudur b = new Mudur();

      b.adi = "Ümit";
      b.soyadi = "Kocabıçak";
      b.departman = "Bilgisayar";

      GenelMudur gm = new GenelMudur();
      gm.adi = "....";
      gm.soyadi = ".....";
      gm.departman = "*****";

      Console.WriteLine(gm.MaasHesapla());
    }

  }
```

[1] Arayüz (Interface) kavramı soyut sınıflar konusunda işlenecektir.
[2] Static kavramı static classlar konusunda işlenecektir.



Programımıza yeni bir sınıf ekleyim GenelMudur sınıfından kalıtım aldırmayı deneyelim.

```
public class EnGenelMudur:GenelMudur
  {

   }
```



sealed sınıflardan kalıtım alınamaz şeklinde bir hata alırız.

5.5. virtual anahtar kelimesi

Temel sınıflarda virtual ile işaretlenmiş yapılar türemiş sınıflara 2 seçenek sunar.

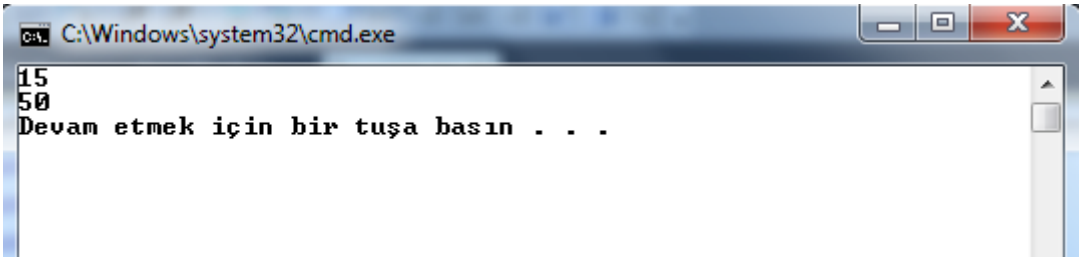
1. Eğer türemiş sınıf bu metodu yazmaz ise kalıtım aldığı sınıfın metodunu kullanır.

2. Türemiş sınıfı bu metodu kendine özgü şekilde yazmak isterse override anahtar kelimesi ile ana sınıfın metodunu geçersiz kılıp(üstüne yazma) kendi yazdığı işlemleri gerçekleştirir.

```
public class GenelSinif
  {
    public double a;
    public double b;

    public virtual void EkranaYazdir()
    {
      Console.WriteLine(a + b);
    }

    public virtual void EkrandaGoster()
    {
      Console.WriteLine(a * b);
    }
  }
```

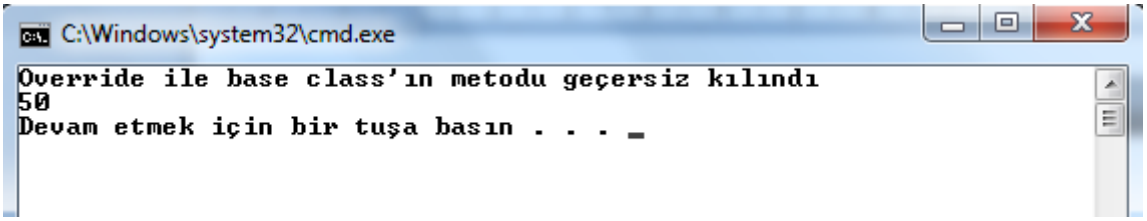
```
class TuremisSinif:GenelSinif
  {
  }
```

```
class Program
  {
    static void Main(string[] args)
    {
      TuremisSinif ts = new TuremisSinif();
      ts.a = 5;
      ts.b = 10;
      ts.EkranaYazdir();
      ts.EkrandaGoster();
    }
  }
```



TuremisSinif kodlarımızı aşağıdaki şekilde değiştirelim.

```
class TuremisSinif:GenelSinif
  {

    public override void EkranaYazdir()
    {
      Console.WriteLine("Override ile base class'ın metodu geçersiz kılındı");
    }
  }
```



Örnek Uygulama: Nokta, daire, küre ve silindirin alanlarını hesaplayan program. (Koordinatlar dikkate alınmayacaktır.)

```
class Nokta
  {
    public double AlanHesapla()
    {
      return 0;
    }
  }
```

```
class Daire
  {
    private double r;
    public Daire(double r)
    {
      this.r = r;
    }
    public double AlanHesapla()
    {
      return Math.PI * r * r;
    }
  }
```

```
class Kure
  {
    private double r;

    public Kure(double r)
    {
      this.r = r;
    }

    public  double AlanHesapla()
    {
      return 4 * Math.PI * r*r;
    }
  }
```

```
class Silindir
  {
    private double r;
    private double h;

    public Silindir(double r, double h)
    {
      this.r = r;
      this.h = h;
    }

    public double AlanHesapla()
    {
      return 2 * Math.PI * r * r + 2 * Math.PI *r * h;
    }
  }
```

```
class Program
  {
    static void Main(string[] args)
    {

      // Yarıçap ve Yükseklik Değerlerinin okunması
      Console.Write("Yarıçap : ");
      double yariCap = Convert.ToDouble(Console.ReadLine());
      Console.Write("Yukselik: ");
      double yukseklik = Convert.ToDouble(Console.ReadLine());

      // Yeni nesnelerin oluşturulması
      Nokta yeniNokta = new Nokta();
      Daire yeniDaire = new Daire(yariCap);
      Kure yeniKure = new Kure(yariCap);
      Silindir yeniSilindir = new Silindir(yariCap, yukseklik);

      // Ekranda Gösterilmesi
      Console.WriteLine("Noktanın Alanı    = {0:F2}", yeniNokta.AlanHesapla());
      Console.WriteLine("Dairenin Alanı    = {0:F2}", yeniDaire.AlanHesapla());
```

```
        Console.WriteLine("Kürenin Alanı     = {0:F2}", yeniKure.AlanHesapla());
        Console.WriteLine("Silindirin Alanı  = {0:F2}", yeniSilindir.AlanHesapla());
    }
}
```

Adım 1: Kodun yeniden kullanılabilirliğinin artırılması ve kodda merkezi yönetim sağlanması.

Silindir 2, Daire ve Küre tek, Nokta sınıfı ise parametresiz kurucu istemektedir. Yeni bir sınıf oluşturarak parametresiz ve 2 parametreli kurucusunu yazalım.

```
class SekilBase
  {
    protected double r;
    protected double h;

    public SekilBase()
    {
    }

    public SekilBase(double r, double h)
    {
      this.r = r;
      this.h = h;
    }

    public virtual double AlanHesapla()
    {
      return 0;
    }
  }
```

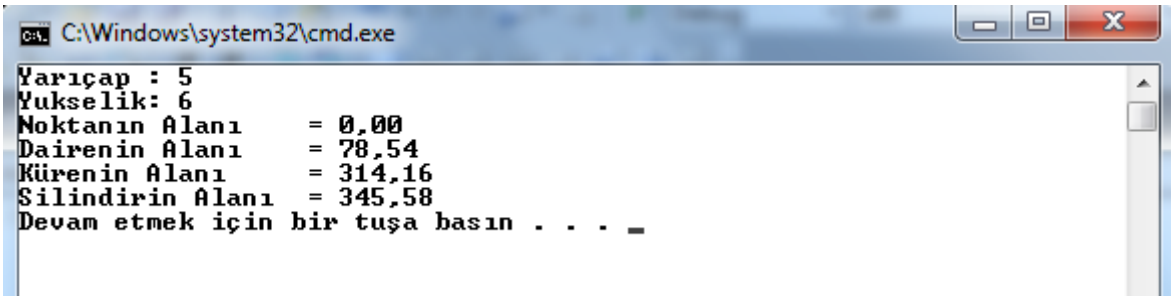
```
class Nokta:SekilBase
  {

  }
```

```
 class Daire:SekilBase
  {
    public Daire(double r):base(r,0)
    {
    }

    public override double AlanHesapla()
    {
      return Math.PI * r * r;
    }
  }
```

```
class Kure:SekilBase
  {

    public Kure(double r):base(r,0)
    {
    }

    public  override double AlanHesapla()
    {
      return 4 * Math.PI * r*r;
    }
  }
```

```
class Silindir:SekilBase
  {

    public Silindir(double r, double h):base(r,h)
    {
    }

    public override double AlanHesapla()
    {
      return 2 * Math.PI * r * r + 2 * Math.PI *r * h;
    }
  }
```

Main tarafındaki kodlarımızı hiç değiştirmeden programı çalıştırdığımızda;



sonucunu elde ederiz.

Adım 2: Main tarafında sürekli olarak alanları ekrana yazdırma kodları yer almaktadır. SekilBase sınıfına ekrana yazdırma kodlarını çekelim. Bu durumda dışardan public olarak belirtilen AlanHesapla() metodununda erişim seviyesini tüm sınıflarda protected düzeye indirerek sarmalamayı da gerçekleştirmiş oluruz. Yeni kodlarımız;

```
class SekilBase
  {
    protected double r;
    protected double h;

    public SekilBase()
    {
    }
```

```
    public SekilBase(double r, double h)
    {
        this.r = r;
        this.h = h;
    }

    protected virtual double AlanHesapla()
    {
        return 0;
    }

    public void EkranaYazdir()
    {
        Console.WriteLine(this.GetType().Name+ " Alanı    = {0:F2}", this.AlanHesapla());
    }
}
```

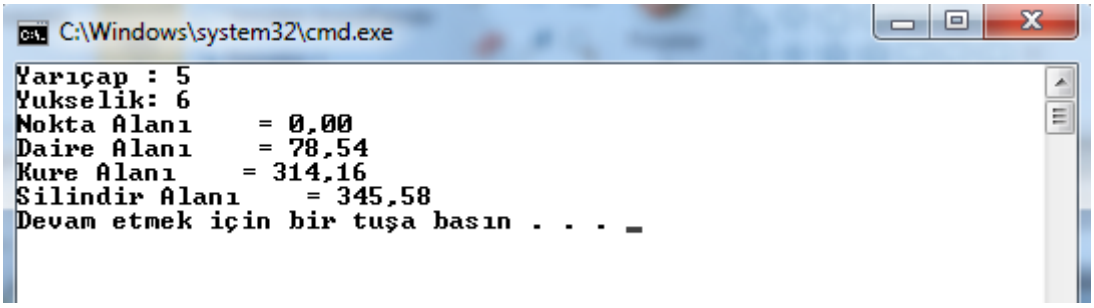
```
class Nokta:SekilBase
  {

  }
```

```
class Daire:SekilBase
  {

    public Daire(double r):base(r,0)
    {
    }
    protected override double AlanHesapla()
    {
        return Math.PI * r * r;
    }
  }
```

```
class Kure:SekilBase
  {

    public Kure(double r):base(r,0)
    {
    }

    protected  override double AlanHesapla()
    {
        return 4 * Math.PI * r*r;
    }
  }
```

```
class Silindir:SekilBase
  {

    public Silindir(double r, double h):base(r,h)
    {
    }

    protected override double AlanHesapla()
    {
      return 2 * Math.PI * r * r + 2 * Math.PI *r * h;
    }
  }
```

```
class Program
  {
    static void Main(string[] args)
    {

      // Yarıçap ve Yükseklik Değerlerinin okunması
      Console.Write("Yarıçap : ");
      double yariCap = Convert.ToDouble(Console.ReadLine());
      Console.Write("Yukselik: ");
      double yukseklik = Convert.ToDouble(Console.ReadLine());

      // Yeni nesnelerin oluşturulması
      Nokta yeniNokta = new Nokta();
      Daire yeniDaire = new Daire(yariCap);
      Kure yeniKure = new Kure(yariCap);
      Silindir yeniSilindir = new Silindir(yariCap, yukseklik);

      // Ekranda Gösterilmesi
      yeniNokta.EkranaYazdir();
      yeniDaire.EkranaYazdir();
      yeniKure.EkranaYazdir();
      yeniSilindir.EkranaYazdir();
    }
  }
```



Adım 3: Main tarafındaki kodlarımız mainin gerççek görevinden uzaklaştırmaktadır. Temel hedeflerimizden biri olan çalışan kodların değiştemeden programımızın gelişebilmesi sağlanamamaktadır. Main içersinde yer alan kodlarımızı başka bir sınıfa(GenelSınıf) çekelim;

```
class GenelSinif
  {
    private double yariCap;
    private double yukseklik;

    public void EkranOku()
    {
      Console.Write("Yarıçap : ");
      yariCap = Convert.ToDouble(Console.ReadLine());
      Console.Write("Yukselik: ");
      yukseklik = Convert.ToDouble(Console.ReadLine());
    }

    public void NesneOlustur()
    {
      Nokta yeniNokta = new Nokta();
      Daire yeniDaire = new Daire(yariCap);
      Kure yeniKure = new Kure(yariCap);
      Silindir yeniSilindir = new Silindir(yariCap, yukseklik);

      // Ekranda Gösterilmesi
      yeniNokta.EkranaYazdir();
      yeniDaire.EkranaYazdir();
      yeniKure.EkranaYazdir();
      yeniSilindir.EkranaYazdir();

    }
  }
```

```
class Program
  {
    static void Main(string[] args)
    {

      GenelSinif gs = new GenelSinif();
      gs.EkranOku();
      gs.NesneOlustur();

    }
  }
```

Main içersinde GenelSinif'ın üyeleriyle muhattap olmamak için GenelSinif içersindeki metotları kurucunun içerisne çekebiliriz.

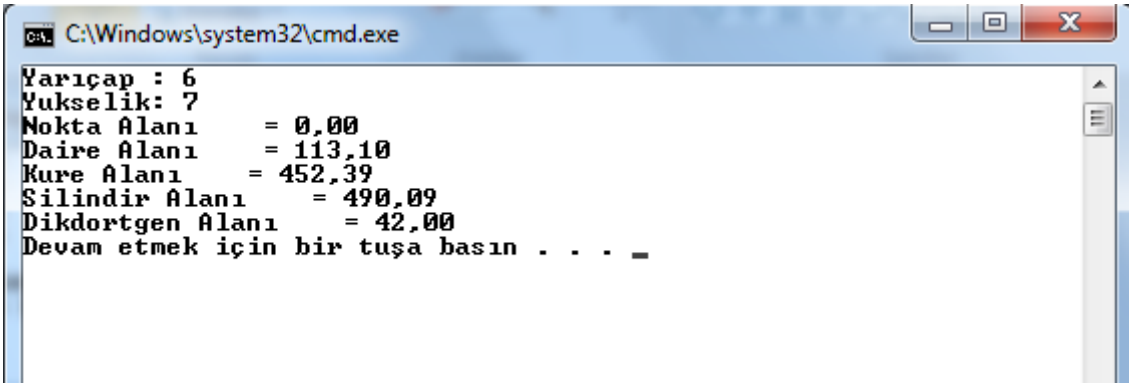
```
class GenelSinif
  {
    private double yariCap;
    private double yukseklik;

    public GenelSinif()
    {
      this.EkranOku();
      this.NesneOlustur();
    }

    private void EkranOku()
    {
      Console.Write("Yarıçap : ");
      yariCap = Convert.ToDouble(Console.ReadLine());
      Console.Write("Yukselik: ");
      yukseklik = Convert.ToDouble(Console.ReadLine());
    }

    private void NesneOlustur()
    {
      Nokta yeniNokta = new Nokta();
      Daire yeniDaire = new Daire(yariCap);
      Kure yeniKure = new Kure(yariCap);
      Silindir yeniSilindir = new Silindir(yariCap, yukseklik);

      // Ekranda Gösterilmesi
      yeniNokta.EkranaYazdir();
      yeniDaire.EkranaYazdir();
      yeniKure.EkranaYazdir();
      yeniSilindir.EkranaYazdir();

    }
  }
```

```
class Program
  {
    static void Main(string[] args)
    {
      GenelSinif gs = new GenelSinif();
    }
  }
```

Adım 4: Yeni bir şeklin(dikdörtgenin) alanın hesaplanmasını sistemimize eklemek istersek;

```
class Dikdortgen:SekilBase
  {
    public Dikdortgen(double en, double boy)
      : base(en, boy)
    {
    }

    protected override double AlanHesapla()
    {
      return r * h;
    }
  }
```

```
class GenelSinif
  {
    private double yariCap;
    private double yukseklik;

    public GenelSinif()
    {
      this.EkranOku();
      this.NesneOlustur();
    }

    private void EkranOku()
    {
      Console.Write("Yarıçap : ");
      yariCap = Convert.ToDouble(Console.ReadLine());
      Console.Write("Yukselik: ");
      yukseklik = Convert.ToDouble(Console.ReadLine());
    }

    private void NesneOlustur()
    {
      Nokta yeniNokta = new Nokta();
      Daire yeniDaire = new Daire(yariCap);
      Kure yeniKure = new Kure(yariCap);
      Silindir yeniSilindir = new Silindir(yariCap, yukseklik);
      Dikdortgen yeniDikdortgen = new Dikdortgen(yariCap, yukseklik); // Dikdörtgen

      // Ekranda Gösterilmesi
      yeniNokta.EkranaYazdir();
      yeniDaire.EkranaYazdir();
      yeniKure.EkranaYazdir();
      yeniSilindir.EkranaYazdir();
      yeniDikdortgen.EkranaYazdir(); //Dikdörtgen

    }
  }
```



Dikkat edilirse Main kodlarımıza ve diğer sınıflarımıza hiçbir değişikliğe gitmeden sistemize yeni şekillerin alanlarını hesaplayan sınıflar ekleyebildik.